JEVIS | Junior Enterprise Pisa

HASTEGA
CODE ENGINEERING

Hackaton Pitch & Build

Progetto V-Shuttle

1. Il contesto e la sfida

Benvenuti. Il cliente di questa simulazione è **Hastega**, una software house con sede a Lucca in forte espansione. Recentemente, abbiamo acquisito una commessa di proporzioni enormi da un cliente DeepTech di altissimo livello: *Waymo LCC*, un'azienda che gestisce flotte di navette a guida autonoma di Livello 4.

Il volume di lavoro richiesto da questo progetto supera la nostra attuale capacità produttiva interna. Per questo motivo, Hastega è alla ricerca di un intero team di sviluppo affiatato, capace e resiliente, a cui affidare l'implementazione completa e la gestione della commessa. Voi siete qui oggi perché il vostro è uno dei team candidati per ottenere questo incarico. Non stiamo valutando solo il vostro codice, ma la vostra capacità di capire un problema reale, progettare un'interfaccia a prova di errore e resistere allo stress.

2. Il Problema del Cliente (Waymo LCC)

Le navette "V-Shuttle" di Waymo LCC operano nei complessi centri storici toscani, districandosi tra Zone a Traffico Limitato (ZTL) e strade strette. Attualmente, il loro software di guida autonoma è estremamente sicuro, ma pecca di eccessiva "timidezza". Di fronte a cartelli stradali complessi, ambigui o degradati, il sistema va in panico e blocca il veicolo.

A bordo della navetta è sempre presente, per obbligo di legge, un "Safety Driver" (chiamiamolo Marco). Marco siede al posto di guida senza toccare il volante e ha davanti a sé un tablet da 12 pollici. Attualmente, quando la navetta si blocca, il tablet mostra a Marco incomprensibili stringhe di log e codici di errore. Marco non sa perché la navetta ha frenato bruscamente, causando peraltro fastidio ai turisti

a bordo (il cosiddetto "Phantom Braking"), e perde secondi preziosi per prendere il controllo manuale tramite joystick.

Waymo LCC ci ha chiesto di risolvere questo scollamento tra la macchina e l'uomo sviluppando due componenti che dovranno dialogare alla perfezione: un "Parser Semantico" e una "Dashboard Touch Live".

3. I Dati e la Logica di Fusione

La navetta è dotata di tre sensori principali: una telecamera frontale (molto affidabile), una telecamera laterale (mediamente affidabile) e un ricevitore V2I che dialoga con le infrastrutture (che a volte può essere offline). Questi tre sensori leggono lo stesso cartello, ma spesso restituiscono testi discordanti o sporchi a causa di errori OCR (ad esempio leggendo "D1V1ET0" al posto di "DIVIETO").

Il vostro primo compito è costruire la logica di backend. Riceverete un file JSON contenente decine di questi scenari. Il vostro algoritmo non dovrà limitarsi a fare una banale media, ma dovrà "fondere" i dati dei tre sensori in modo intelligente per capire quale sia la lettura reale. Una volta normalizzato il testo, il sistema dovrà parsarlo per capire se si tratta di un divieto, se ci sono eccezioni (ricordate che la navetta è considerata un "BUS", quindi se c'è scritto "Eccetto BUS", la navetta può passare) o se ci sono vincoli legati a orari e giorni festivi. In base a questo, il sistema deciderà l'azione da intraprendere: STOP, GO, oppure richiedere l'intervento di Marco se la confidenza finale è troppo bassa.

4. L'Interfaccia Utente (Dashboard Live)

La vera sfida UX è creare l'interfaccia per Marco. Dimenticatevi schermate fitte di dati: Marco ha al massimo due secondi per capire la situazione e reagire mentre si trova su un veicolo che potrebbe star frenando bruscamente.

La vostra Dashboard non deve essere una tabella statica in cui si clicca per andare avanti, ma un vero e proprio **simulatore continuo**. Dovrà includere un pulsante "START SIMULATION" che caricherà l'intero dataset e avvierà un ciclo automatico: il sistema farà scorrere e valuterà *tutti* gli scenari del file JSON in sequenza, passando da uno all'altro automaticamente ogni 4 secondi senza alcun bisogno di interazione da parte dell'utente.

Se il vostro algoritmo rileva un'incertezza e richiede la conferma umana, questo loop automatico si fermerà temporaneamente: Marco avrà esattamente 2 secondi per premere un pulsante gigante di "OVERRIDE" o "CONFERMA". Se il tempo scade senza intervento umano, il sistema dovrà tutelarsi attivando una frenata d'emergenza automatica (impostando l'azione su STOP) per poi riprendere il ciclo con lo scenario successivo. L'interfaccia deve parlare la lingua dei colori: rosso per fermarsi, verde per procedere, giallo per attenzione. Nessun gergo tecnico deve apparire a schermo.

5. Regole d'Ingaggio e Lo Stress Test Finale

Siete liberi di utilizzare qualsiasi stack architetturale: Python, NodeJS, React, Vanilla JS, ecc. L'uso dell'Intelligenza Artificiale (ChatGPT, Claude, Copilot) è caldamente incoraggiato per generare componenti boilerplate, CSS o aiutarvi con le espressioni regolari. **Tuttavia, c'è un limite invalicabile:** non potete inviare il testo del cartello alle API di OpenAI per far decidere al modello linguistico cosa fare. L'intelligenza decisionale, l'algoritmo di fusione e il parsing semantico devono risiedere nel vostro codice in modo algoritmico e deterministico.

Durante l'hackathon avrete a disposizione dei **Tutor**. Usateli. Se la traccia vi sembra ambigua o siete bloccati su un requisito logico, fare domande mirate è segno di professionalità, non di debolezza.

Infine, preparatevi al vero test della vostra architettura. Avrete a disposizione il file fin dall'inizio per costruire il sistema. Ma attenzione: **esattamente 10 minuti prima della scadenza del tempo**, vi verrà consegnato un altro file segreto contenente 30 nuovi scenari inediti con casi limite complessi. Dovrete caricare questo file nel vostro sistema e dimostrare che il vostro codice è sufficientemente solido da gestirli senza crashare. Questo è il momento in cui i sistemi scritti male crollano.

6. La Giuria

Il vostro lavoro sarà valutato da tre referenti chiave di Hastega, ognuno con un focus specifico:

- **Paolo (Tech Lead):** Analizzerà il vostro codice, l'algoritmo di fusione dei sensori e la robustezza del backend, in particolare durante lo stress test finale.
- **Francesco (Product Manager):** Valuterà l'implementazione lato utente. Si concentrerà sull'interfaccia di Marco, l'usabilità, i flussi di navigazione e il rispetto della regola della "Simulazione Live".
- **Andrea (Recruiter):** Vi osserverà durante tutta la prova. Valuterà le vostre dinamiche di team, come vi dividete i compiti e come presentate il progetto.

7. Modalità di Consegna

Per essere ammessi alla valutazione finale, la consegna deve avvenire **esclusivamente tramite un repository GitHub**.

Il repository **DEVE ESSERE PUBBLICO**. Qualsiasi consegna effettuata tramite repository privato, file .zip, pendrive o link non accessibili comporterà l'**INVALIDAZIONE IMMEDIATA** della prova.

L'unico requisito strutturale per il repo è la presenza di un file `README.md` ben curato che contenga: una breve descrizione del progetto, un paio di screenshot della dashboard e, soprattutto, l'elenco esatto dei comandi da lanciare nel terminale per far partire il vostro progetto.

Il repo dovrà essere condiviso con le mail p.micheletti@hastega.it, f.romeo@hastega.it, a.dellamaggiore@hastega.it.

Al termine della prova ogni team avrà 90 secondi per spiegare a voce l'implementazione effettuata.

8. Criteri di Valutazione (100pt)

La giuria assegnerà il punteggio basandosi su 30 criteri di accettazione rigidi, suddivisi nelle seguenti quattro categorie.

A. UX Dashboard

1. **Presenza Simulazione:** Esiste un bottone "START SIMULATION" chiaramente visibile.
2. **Automazione:** Il sistema avanza automaticamente al prossimo scenario ogni 4 secondi.
3. **Gestione Timeout:** In caso di richiesta di conferma, è presente un timer visibile di 2 secondi per l'operatore.
4. **Fallback di Sicurezza:** Se il timer di 2 secondi scade senza interazione, il sistema imposta l'azione su "STOP" in automatico.
5. **Ergonomia (Fat Finger):** I pulsanti di interazione (es. OVERRIDE) sono sufficientemente grandi da essere premuti su un tablet in movimento.
6. **Leggibilità Cromatica:** Uso corretto della psicologia dei colori (Verde=Go, Rosso=Stop) senza ambiguità.
7. **Trasparenza Dati:** La dashboard mostra chiaramente il testo "fuso" finale che il sistema ha compreso, non i log grezzi.
8. **Feedback Confidenza:** Il livello di confidenza della lettura è mostrato in modo visivo e immediato (es. barre di riempimento, non solo numeri decimali).
9. **Assenza di Gergo:** Non compaiono mai a schermo log di errore tecnici, eccezioni di codice o messaggi indecifrabili per un utente non tecnico.
10. **Reattività:** L'interfaccia si aggiorna istantaneamente ai click dell'operatore senza lag o ricaricamenti di pagina.

B. Robustezza Algoritmica

1. **Test Segreto (Criterio d'Oro):** L'algoritmo elabora correttamente i 30 scenari segreti forniti a T-10 minuti senza andare in crash.
2. **Logica di Fusione:** Implementazione di un algoritmo di fusione sensori che tiene conto dell'affidabilità variabile delle 3 fonti (non è una semplice media).
3. **Gestione Sensori Offline:** Il sistema non va in errore se il `V2I_receiver` o un altro sensore restituisce valori `null`.
4. **Normalizzazione OCR:** Presenza di un modulo che pulisce efficacemente le stringhe sporche (es. converte "D1V1ET0" in "DIVIETO") prima della valutazione.
5. **Logica Temporale:** Corretta interpretazione di orari di inizio/fine (es. ZTL) incrociati con l'ora attuale dello scenario.

6. **Logica Calendario:** Corretta identificazione e gestione dei "Giorni Festivi" o regole legate ai giorni della settimana.
7. **Eccezioni Logiche:** Il sistema riconosce e applica correttamente la regola "Eccetto BUS" (o varianti affini) forzando lo stato GO.
8. **Deterministico:** L'algoritmo non fa chiamate esterne (API OpenAI/LLM) per valutare semanticamente il testo durante l'esecuzione.
9. **Output Strutturato:** Il sistema produce internamente il JSON di output "Legacy" esatto richiesto dal cliente (action, reason, confidenza, etc.).
10. **Clean Code:** Il codice backend/logico è modularizzato (es. funzioni separate per pulizia testo, fusione e decisione) e non è un singolo script monolitico.

C. Documentazione

1. **Accessibilità Repo:** Il repository GitHub è PUBBLICO e accessibile dalla giuria senza richiedere permessi.
2. **README Base:** Presenza del file `README.md` nella root del progetto.
3. **Descrizione Progetto:** Il README contiene un paragrafo chiaro che spiega qual è il problema risolto e l'approccio scelto.
4. **Visuals:** Sono presenti almeno due screenshot integrati nel README che mostrano la Dashboard in stati diversi.
5. **Setup Infallibile:** I comandi per installare le dipendenze (es. `npm install`, `pip install -r`) sono scritti chiaramente.
6. **Run Instructions:** I comandi per avviare il server/applicazione (es. `npm run dev`, `python app.py`) funzionano al primo tentativo.
7. **Spiegazione Fusione:** È presente un paragrafo (o un file a parte) che descrive a parole la formula matematica/logica usata per fondere i 3 sensori.
8. **Mappatura Edge Cases:** Documentazione delle scelte fatte per gestire i casi limite (cosa succede se 2 sensori dicono A e 1 dice B?).
9. **Schema Architetturale:** Presenza di un semplice schema (anche testuale o generato via tool come Mermaid) che mostra come comunicano backend e frontend.
10. **Pulizia Repo:** Assenza di cartelle inutili caricate per errore (es. `node_modules`, `.env`, file temporanei di sistema).